

Multi-agent Debugging — Worked Example: Solution

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)
- [Real cause vs. symptom fix](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Basic

Real cause: a plain typo — "Reserach" instead of "research_agent" — in whatever logic produces the routing string. `Command(goto=...)` has no fuzzy matching; it has to name an exact node in the graph, or the graph fails immediately and clearly.

The fix:

```
def decide_next_agent(state):
    if needs_research(state):
        return "research_agent"
    if needs_analysis(state):
        return "analysis_agent"
    return "writer_agent"
```

Match the string exactly to the node name used in `graph.add_node("research_agent", research_node)`.

Test that would have caught it:

```
def test_supervisor_routes_to_a_real_node_name(compiled_graph):
    real_node_names = set(compiled_graph.get_graph().nodes)
    for possible_target in decide_next_agent.possible_return_values:
        assert possible_target in real_node_names
```

The error message here already names every available node — the fastest read is to compare the misspelled name against that list character by character, rather than re-reading the routing logic first.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Intermediate

Real cause: `research_node` writes its result to a shared-state key (`finding`) that the `SharedState` schema never declared and no other node reads from — the schema declares `research_findings` instead. LangGraph doesn't reject an update to an undeclared key by default, so this fails silently: the Research agent thinks it succeeded (it did write *something*), and the Analysis agent thinks it correctly found nothing (it did read `research_findings` , which genuinely is empty) — both are technically right about their own narrow view, and the mismatch only exists between them.

The fix:

```
class SharedState(TypedDict):
    task: str
    research_findings: str
    analysis: str
    draft: str

def research_node(state):
    finding = run_research_tools(state["task"])
    return Command(goto="analysis_agent", update={"research_findings": finding})
```

Use the same field name the schema declares, everywhere a node writes to or reads from that field.

Test that would have caught it:

```
def test_research_agent_writes_the_field_analysis_agent_reads(fake_research_tools):
    fake_research_tools.return_value = "The refund policy allows returns within 30 days."
    result = research_node({"task": "refund policy"})
    written_keys = set(result.update.keys())
    assert "research_findings" in written_keys
```

Checking the *exact keys* a node's `Command.update` writes, against the schema's declared field names, catches a silent naming drift that "did the pipeline finish without an error" never would — this bug produces no exception anywhere.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Real-world

Real cause: the two specialist descriptions genuinely overlap for any task that needs both finding *and* interpreting information — there's no rule telling the supervisor which one wins when both apply, so its routing decision (an LLM call) is sensitive to small wording differences in the task, and comes out differently on essentially identical requests. This is Doc11's own "ambiguous routing" edge case, and it's a real production risk specifically because it looks like nondeterminism in the model, when the actual fix is a design gap in how the specialists were defined.

The fix:

```
SPECIALISTS = {
    "research_agent": "Finds NEW factual information the system doesn't already h
    "analysis_agent": "Interprets and draws conclusions from information that has
}
```

Sharpening each description so their responsibilities are mutually exclusive — instead of both plausibly covering the same task — removes the ambiguity at its source. For a task that genuinely needs both, the fix is structural: route to Research first, then have the supervisor route its output to Analysis next, rather than asking one routing decision to pick a single winner between two valid answers.

Test that would have caught it:

```
def test_supervisor_routes_consistently_for_equivalent_task_wordings():
    same_task_worded_differently = [
        "find and interpret last quarter's revenue numbers",
        "look up and interpret last quarter's revenue numbers",
        "get and interpret last quarter's revenue figures",
    ]
    routes = []
    for task in same_task_worded_differently:
        routes.append(supervisor_decide({"task": task}))
    assert len(set(routes)) == 1
```

Running several *equivalent* task phrasings through the supervisor and checking they route the same way is the only way to catch ambiguous routing — a test with one fixed wording per task can't surface a problem that's specifically about wording sensitivity.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Multi-agent

Real cause: Round 2's field-name bug was patched by *adding* a new field (`finding`) instead of renaming the old one everywhere it was used — so the schema ended up with two fields that look like they mean the same thing, one live (`finding` , used by Research and Analysis) and one dead (`research_findings` , still declared, still read by the Reviewer, never written by anyone). Every pair-wise test happened to use whichever field its own author picked, so both pairs "worked" in isolation while disagreeing about which field is the real one — a disagreement that only becomes visible once all 4 nodes run together against the one real, shared state object.

The fix:

```
class SharedState(TypedDict):
    task: str
    finding: str
    analysis: str
    draft: str
    review_passed: bool
    revision_count: int

def reviewer_node(state):
    if state["finding"] not in state["draft"] and state["analysis"] not in state["draft"]:
        return Command(goto="writer_agent", update={
            "review_passed": False,
            "revision_count": state["revision_count"] + 1,
        })
    return Command(goto=END, update={"review_passed": True})
```

One field name, used consistently by every node that reads or writes it, removes the dead field entirely instead of leaving two that look interchangeable. (This example also fixes the Reviewer's inverted check — `not in` on an always-empty string used to make the rejection condition always true regardless of the field-name bug; worth noting as a second, separate mistake stacked on the first.)

Test that would have caught it:

```
def test_full_pipeline_produces_an_approved_draft(fake_research_tools, fake_search_tools):
    result = run_full_pipeline({"task": "summarize Q1 revenue"})
```

```
assert result["review_passed"] is True
assert result["revision_count"] <= 2
```

This has to run all 4 nodes together, against one real shared state object passed hand to hand — not 2 separate pair tests each with their own fixture. A schema-level check helps too:

```
def test_every_node_reads_and_writes_only_declared_schema_fields():
    declared_fields = set(SharedState.__annotations__.keys())
    for node_function in [research_node, analysis_node, writer_node, reviewer_node]:
        for field_name in fields_read_or_written_by(node_function):
            assert field_name in declared_fields
```

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Real cause vs. symptom fix

A symptom fix at Round 2 would have been exactly what actually happened at Round 4 in this story: add a new field instead of renaming the old one, so the one broken call site stops erroring, without checking whether anything *e/se* still depends on the old name. That single shortcut is what turned a one-line bug into a 4-agent, un-reproducible-in-isolation failure two rounds later — a direct illustration of Core Concepts' warning that a symptom fix doesn't just fail to solve the problem, it can make the *next* occurrence more confusing than the first. The real-cause fix at every round here is the same kind of move: make the two sides of a handoff — a routing string and a node name, a writer and a reader of one state field, two specialist descriptions and one routing decision — agree by construction, not by every author independently guessing correctly.